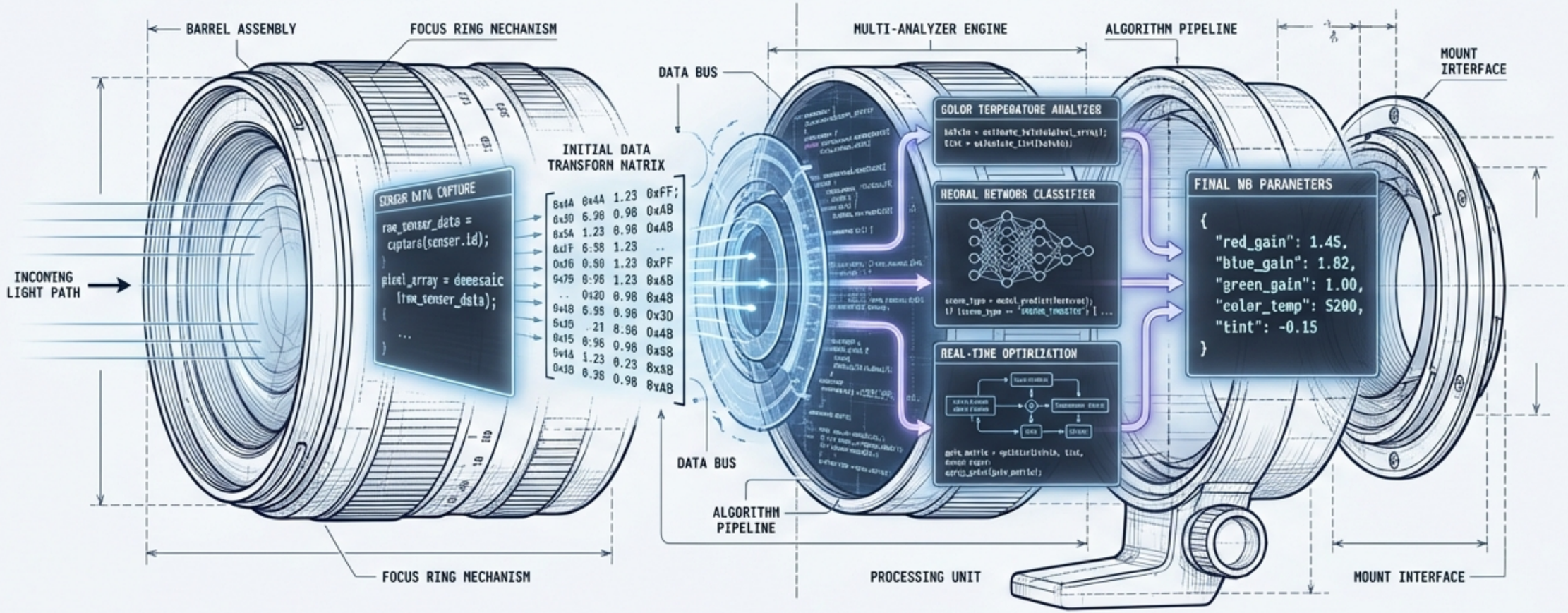
Chapter 7: Building an Intelligent Auto White Balance System



A Deep Dive into Multi-Analyzer Architectures and Real-Time Implementation

Implementation Reference: github.com/JeffRen1977/auto-white-balance

NotebookLM

The Challenge.



Estimating true illumination in a chaotic world



The Core Challenge

How do we estimate true scene illumination when every light source is different? Traditional "Gray World" assumptions fail when the scene is dominated by a single color.

The Chapter 7 Solution

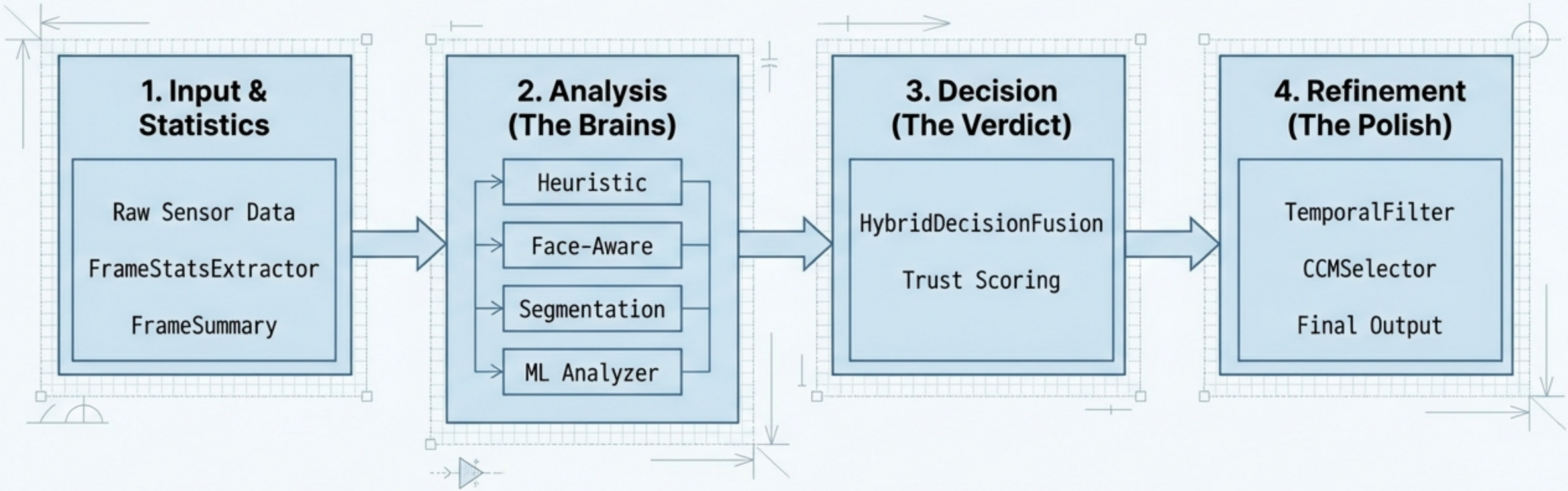
A **Multi-Analyzer Framework**. Instead of relying on one method, we deploy a “Committee of Experts” that votes on the correct white point.

Key System Features

- Hybrid Logic: Heuristic + Face-aware + Segmentation + ML
- Trust-Based Fusion: Resolves conflicts between analyzers
- Temporal Smoothing: Flicker-free video stability
- Real-Time Optimized: >33 FPS on mobile

NotebookLM

Mapping the pipeline: From sensor data to corrected image



The system is modular. Analyzers can be swapped or upgraded without breaking the fusion or refinement logic.

NotebookLM

The Evidence Package: Anatomy of a FrameSummary

Before the camera can "think," it needs evidence. The **FrameSummary** dataclass abstracts raw sensor data into a format all analyzers can read.

The Grid Stats

Scene Context

Semantic Targets

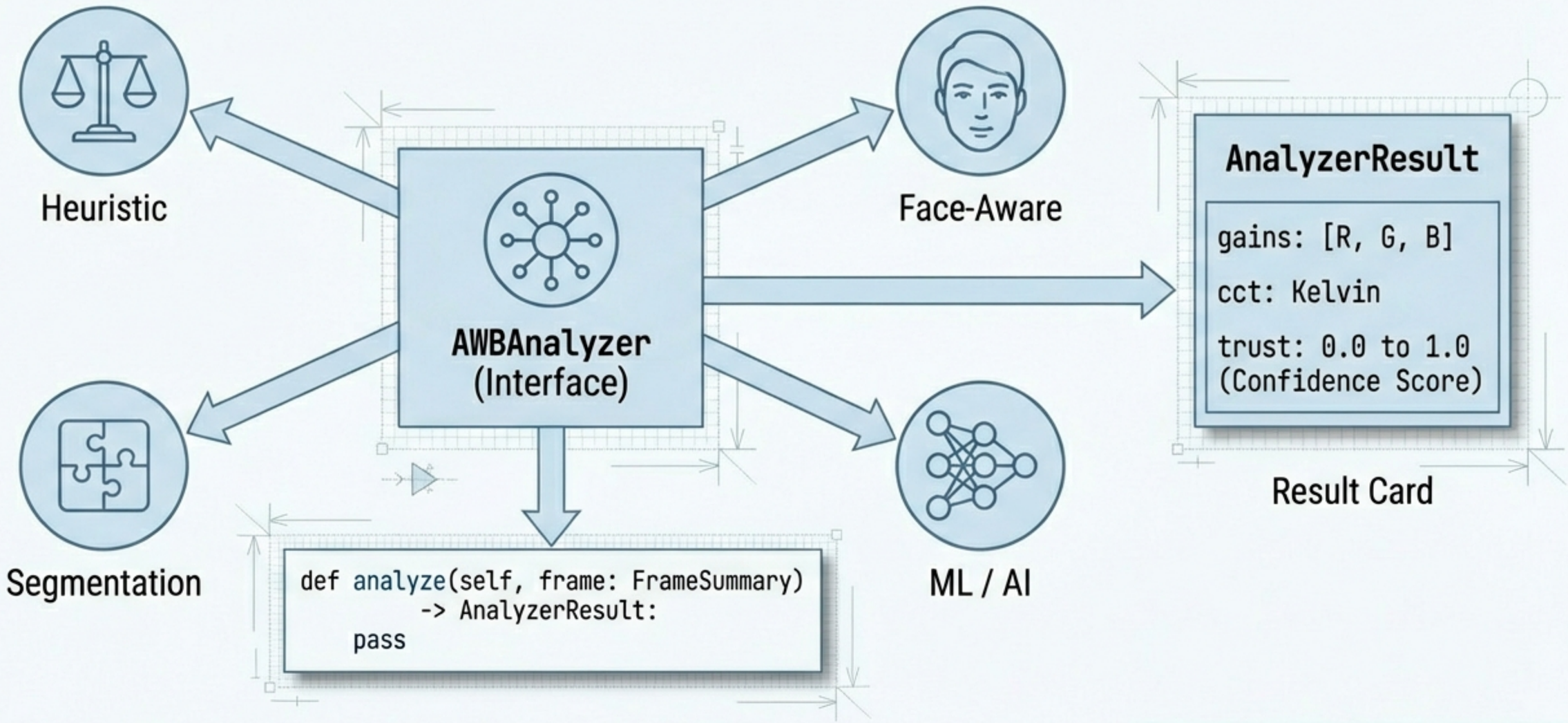
```
@dataclass
class FrameSummary:
    # Statistical Data
    r_grid: np.ndarray  # 64x48 stats
    g_grid: np.ndarray
    b_grid: np.ndarray

    # Metadata
    bv: float           # Brightness Value
    timestamp: float

    # Semantic Data
    scene: str          # e.g., 'sunset'
    face_rois: List     # Face coordinates
    segmentation_map: np.ndarray
```


NotebookLM

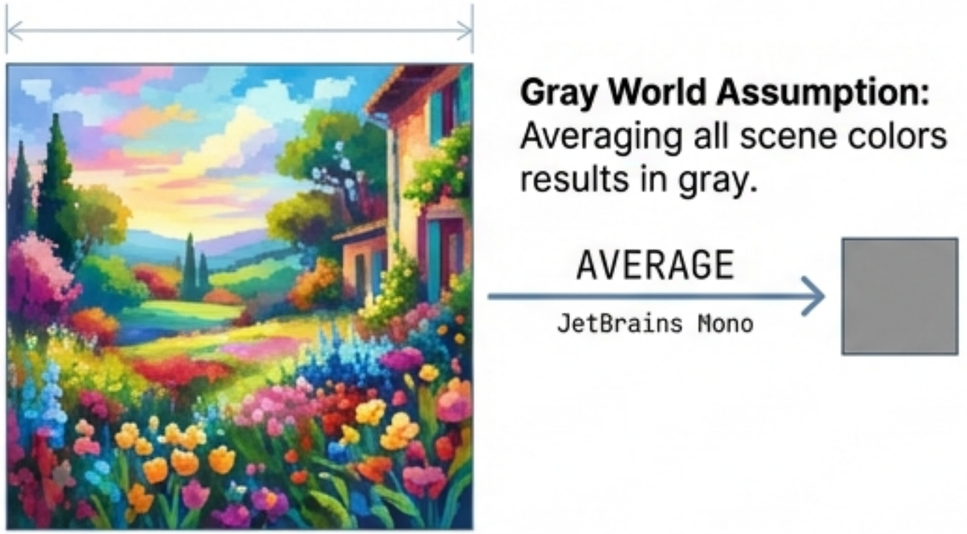
The Interface: A plug-and-play analyzer architecture



NotebookLM

Heuristics and ML: Balancing speed with learned intuition

Analyzer 1: Heuristic (The Veteran)



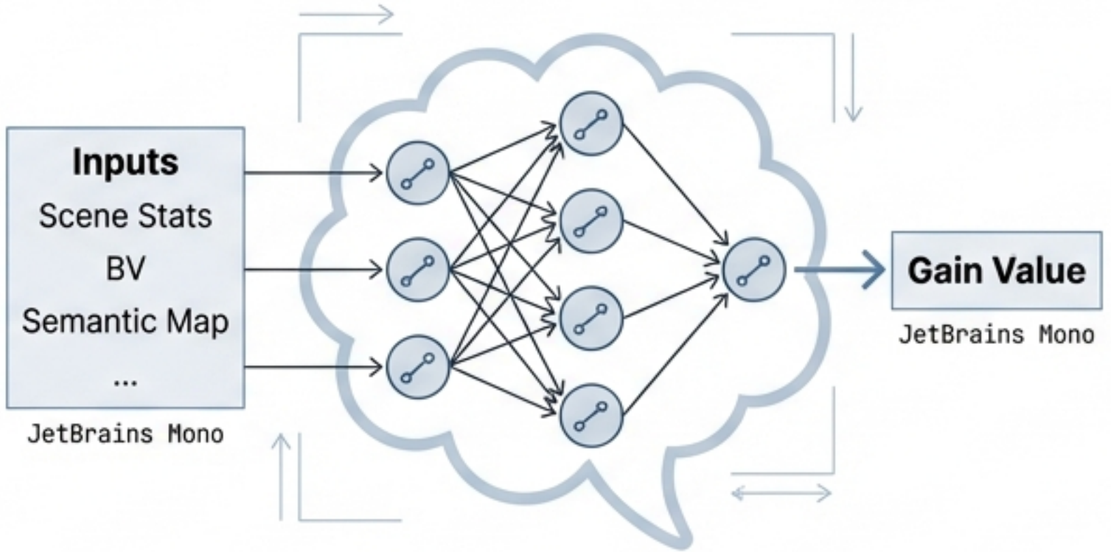
Gray World Assumption:
Averaging all scene colors results in gray.

AVERAGE

JetBrains Mono

- **Logic:** Gray World Assumption / Daylight Locus
- **Best For:** General outdoor scenes, consistent lighting
- **Pros:** Extremely fast execution

Analyzer 2: Machine Learning (The Specialist)

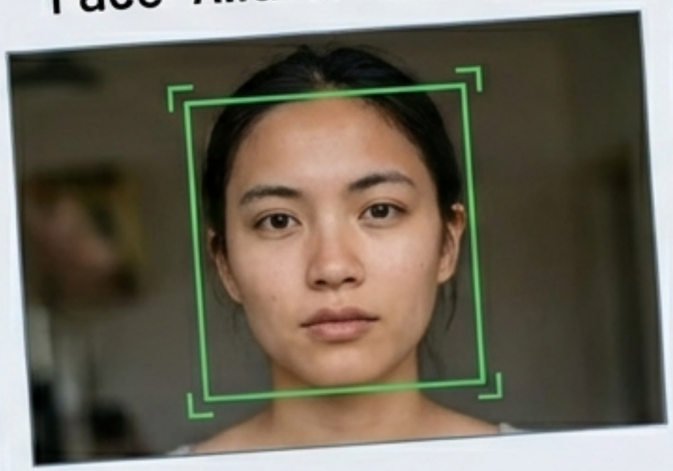


- **Logic:** Trained models for complex mixed lighting
- **Trigger:** Low Light conditions (BV < 2.0)
- **Mechanism:** Infers gains when signal-to-noise ratio confuses standard stats

NotebookLM

Semantic Awareness: Prioritizing faces and context

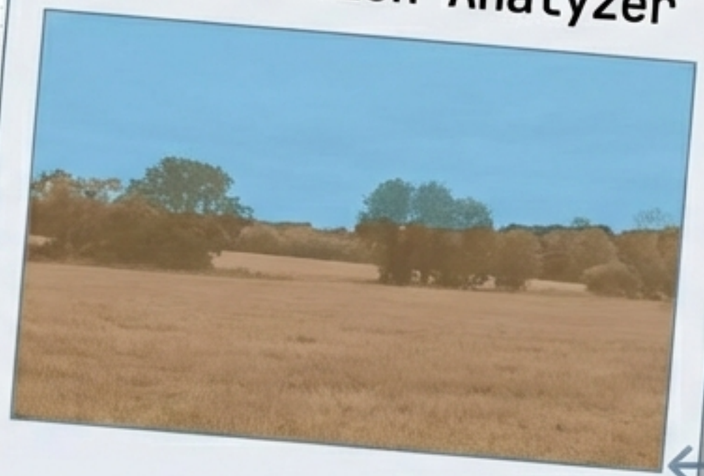
Face-Aware Analyzer



- **Priority**: High. Humans define image quality by skin tone.
- **Mechanism**: Extracts stats only from 'face_rois'.
- **Target**: Optimizes for skin tone preference (0.48).

ROI DATA FLOW

Segmentation Analyzer



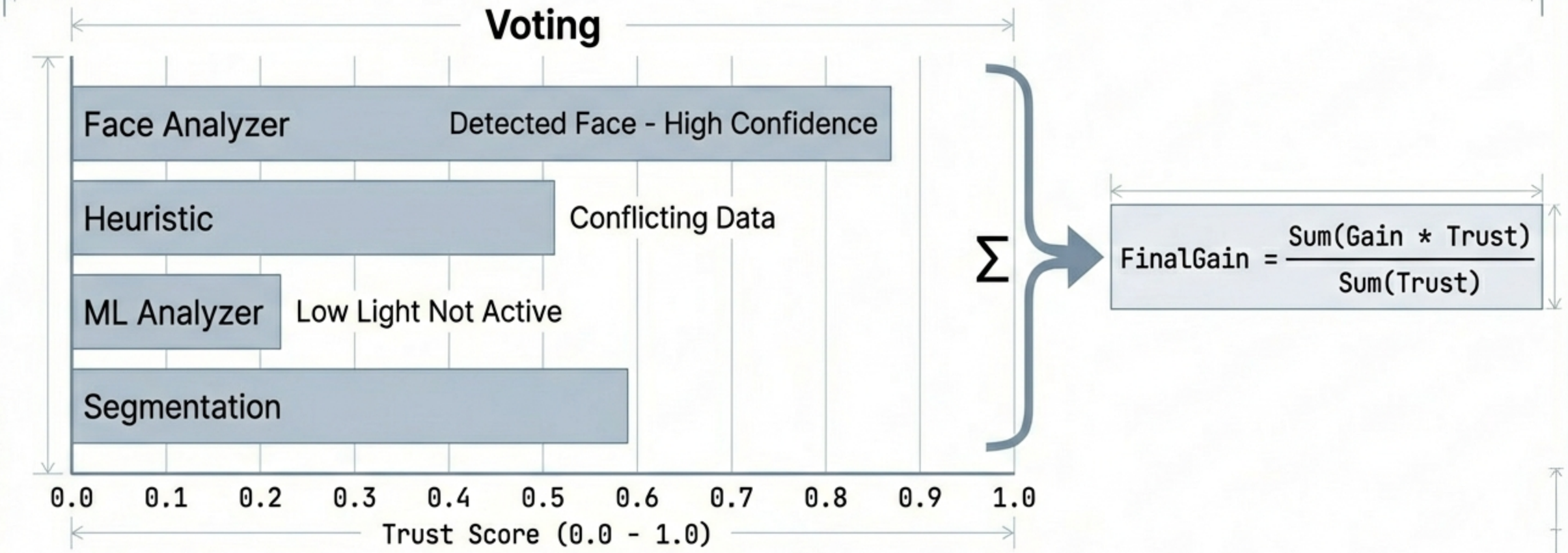
PIXEL WEIGHT MAP

- **Logic:** Not all pixels are equal.
- **Weights:**
  - Person/Skin: 0.8 (High)
  - Sky: 0.6 (Medium)
  - Background: 0.3 (Low)

NotebookLM

The Boardroom Meeting: Trust-based fusion logic

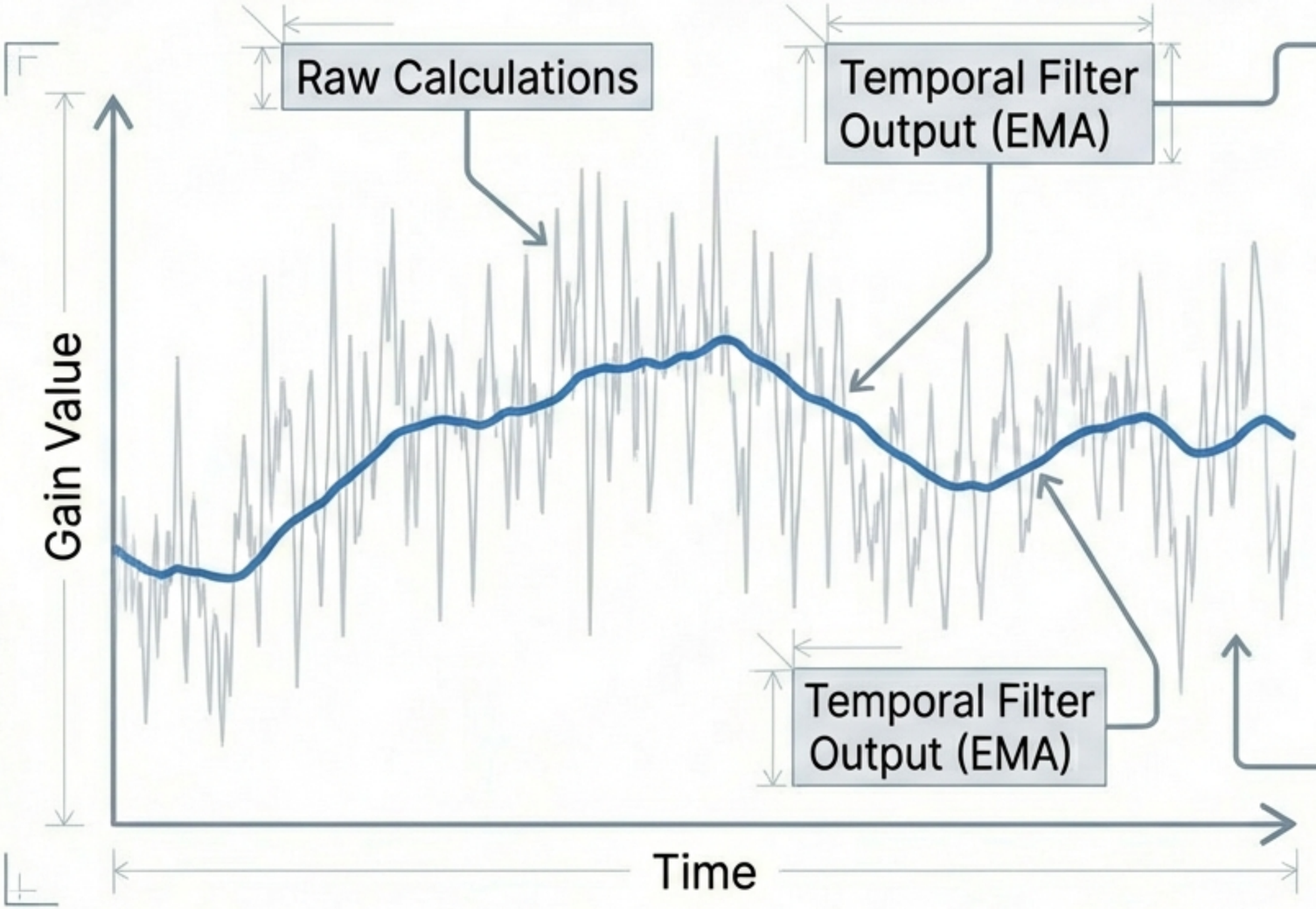
Solving the conflict between analyzers.



High-trust inputs dominate the final result, filtering out low-confidence outliers.

NotebookLM

Refinement: Temporal Filtering for Video



Class: `TemporalFilter`
Method: `Exponential Moving Average (EMA)`

Parameters:

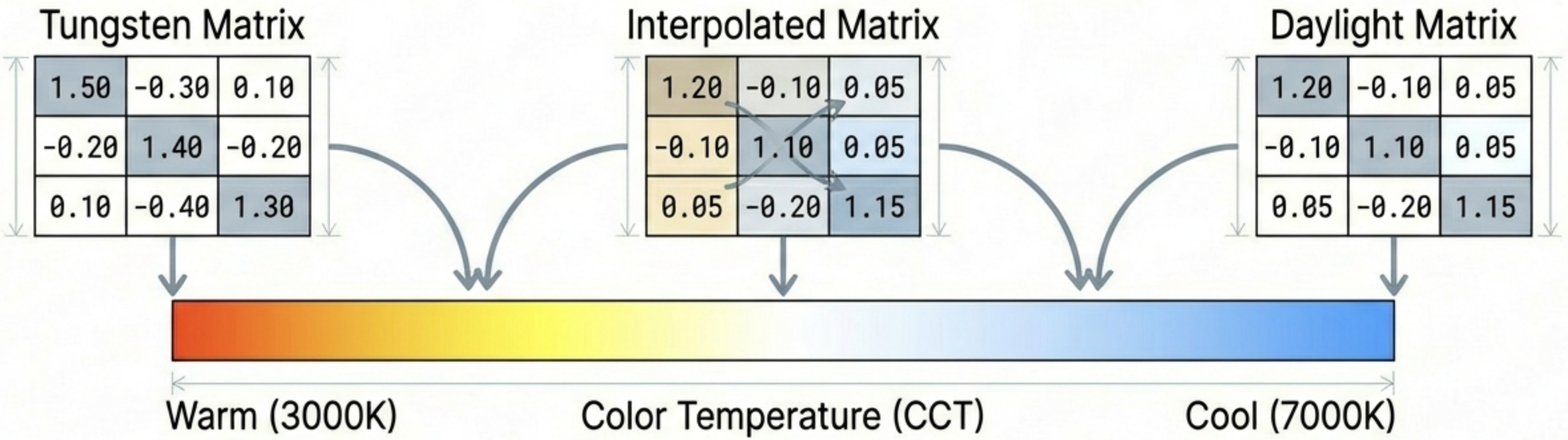
- `alpha_gains` (`0.3`): Controls convergence speed.
- `alpha_cct` (`0.2`): Controls color temp smoothness.

Result: Eliminates video **“pulsing”** and color shifts.

NotebookLM

Advanced Color: CCM Selection

White balance sets the neutral point. The Color Correction Matrix (CCM) fixes the relationship between colors (Hue/Saturation).



Adaptive Factors:

- **CCT:** Interpolate between Tungsten and Daylight.
- **Scene:** Portrait (Warm bias) vs. Landscape (Cool bias).
- **Light Level:** Low light adjustment.

NotebookLM

Configuration: Real-Time vs. Quality



Scenario A: Real-Time (Mobile Preview)

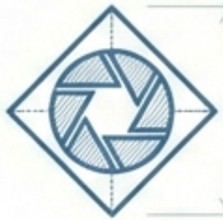
Goal: Speed (<10ms) & Responsiveness.

Grid: Subsampled (64x48).

Active Analyzers: Heuristic + Face Only.

Temporal Alpha: 0.5 (Fast convergence).

JetBrains Mono



Scenario B: High Quality (Capture)

Goal: Maximum Fidelity.

Grid: Full Resolution.

Active Analyzers: Heuristic + Face + Seg + ML (All).

Temporal Alpha: 0.2 (Smooth stability).

JetBrains Mono

NotebookLM

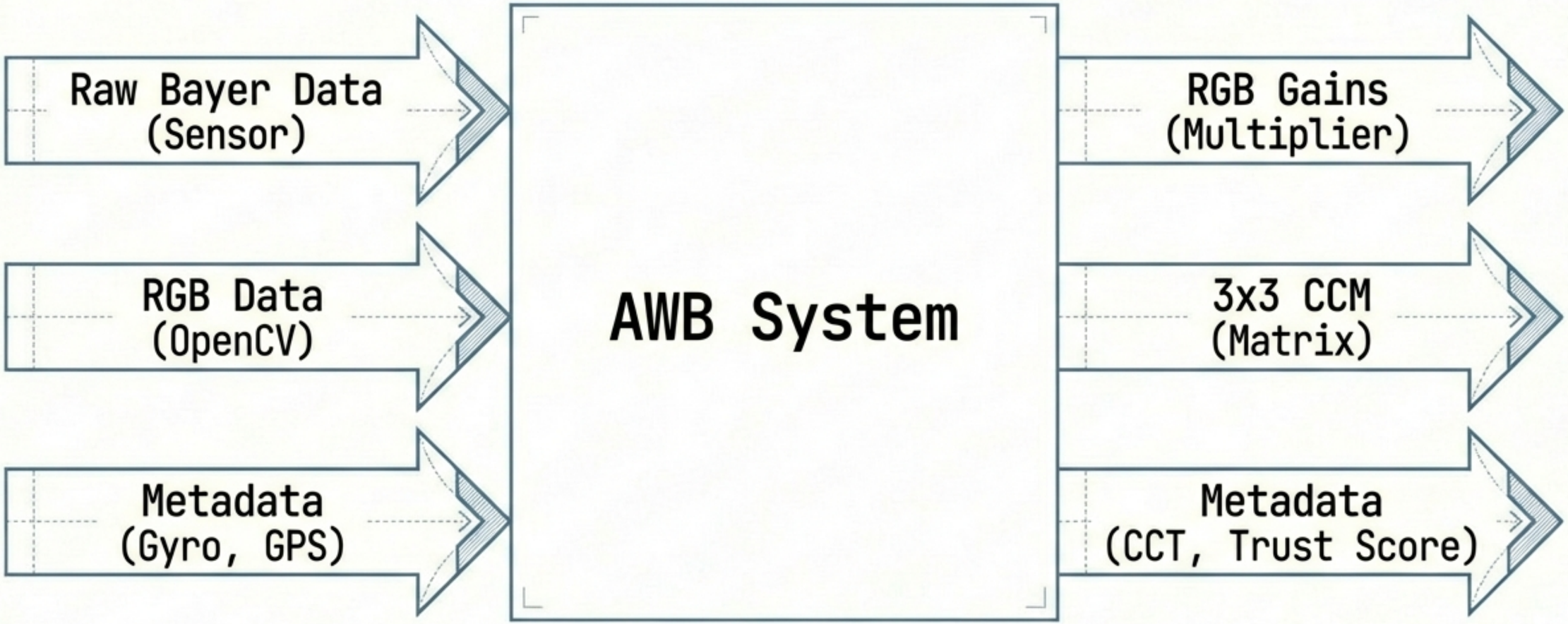
Implementation Details: The Code Structure

```
class AutoWhiteBalanceSystem:
    1. stats = stats_extractor.extract(image)   ← Mining Data
    2. results = [analyzer.analyze(stats)...]   ← Committee Vote
    3. fused = fusion.compute(results)          ← Decision
    4. smooth = temporal_filter.update(fused)   ← Stabilization
    5. ccm = ccm_selector.select(smooth)        ← Color Polish
```

Extensibility: Inherit from AWBAnalyzer to add custom algorithms without touching the core loop.

NotebookLM

Integration & Input Sources

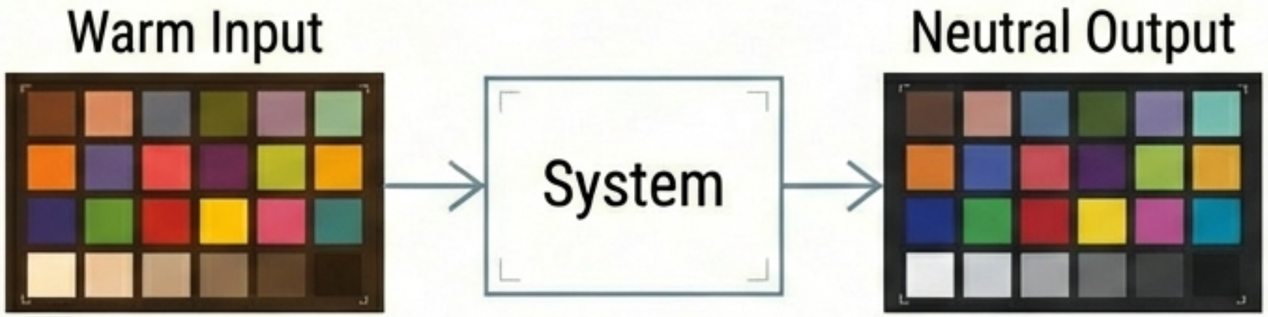


Hardware Agnostic: Designed to sit between the sensor driver and the ISP pipeline.

NotebookLM

Testing & Validation

Synthetic Testing



Performance Monitoring



Trust Analysis

Ensures the fusion logic isn't biased toward one specific algorithm.



NotebookLM

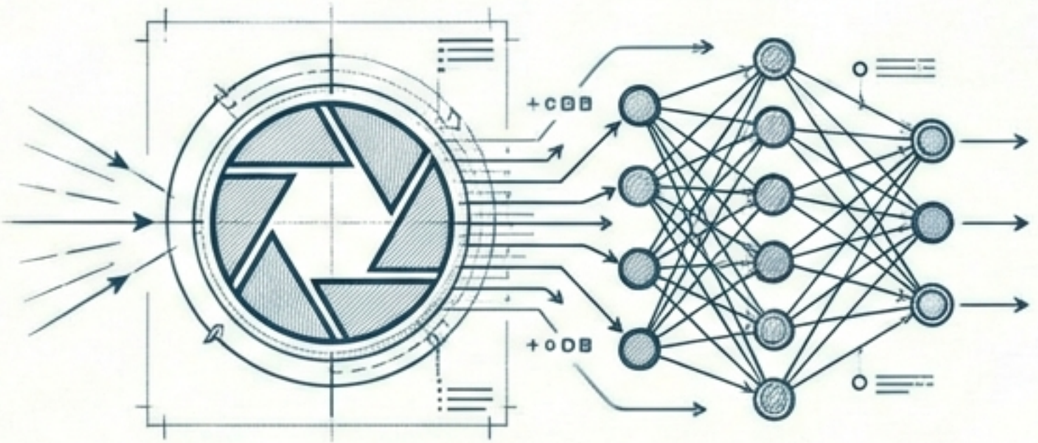
Summary & Future Directions

Inter Tight

- **Recap:** We moved from “Gray World” theory to a concrete, code-based Multi-Analyzer System.
- **Key Takeaway:** The power lies in Fusion. A weighted system creates professional-grade reliability.

The Future Scope

- **End-to-End Neural AWB:** Replacing the pipeline with a single Deep Learning network.
- **Personalized Learning:** Systems that learn user preference (e.g., “I prefer warmer sunsets”).



NotebookLM